Tasks on K8S-05

1. Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

Objective: Create a namespace and restrict the maximum number of Pods to 3 and Services to 2.

Implementation Steps:

1. Check the existing namespaces:

   kubectl get namespaces

2. Create the namespace:

   kubectl create namespace dev-environment

3. Verify the namespace:

   kubectl get namespaces

4. Create a ResourceQuota YAML file:

   vi dev-quota.yaml

5. Add the following configuration:

   apiVersion: v1

   kind: ResourceQuota

   metadata:

    name: dev-resource-quota

    namespace: dev-environment

   spec:

    hard:

     pods: "3"

          services: "2"

6. Save the file and apply the ResourceQuota:

   kubectl apply -f dev-quota.yaml

7. Verify the ResourceQuota:

   kubectl get resourcequota -n dev-environment

8. Describe the ResourceQuota:

kubectl describe resourcequota dev-resource-quota -n dev-environment

9. Check the namespace details:

   kubectl describe namespace dev-environment

10. Create three test Pods to verify the quota:

    kubectl run dev-pod-1 --image=nginx -n dev-environment

    kubectl run dev-pod-2 --image=nginx -n dev-environment

    kubectl run dev-pod-3 --image=nginx -n dev-environment

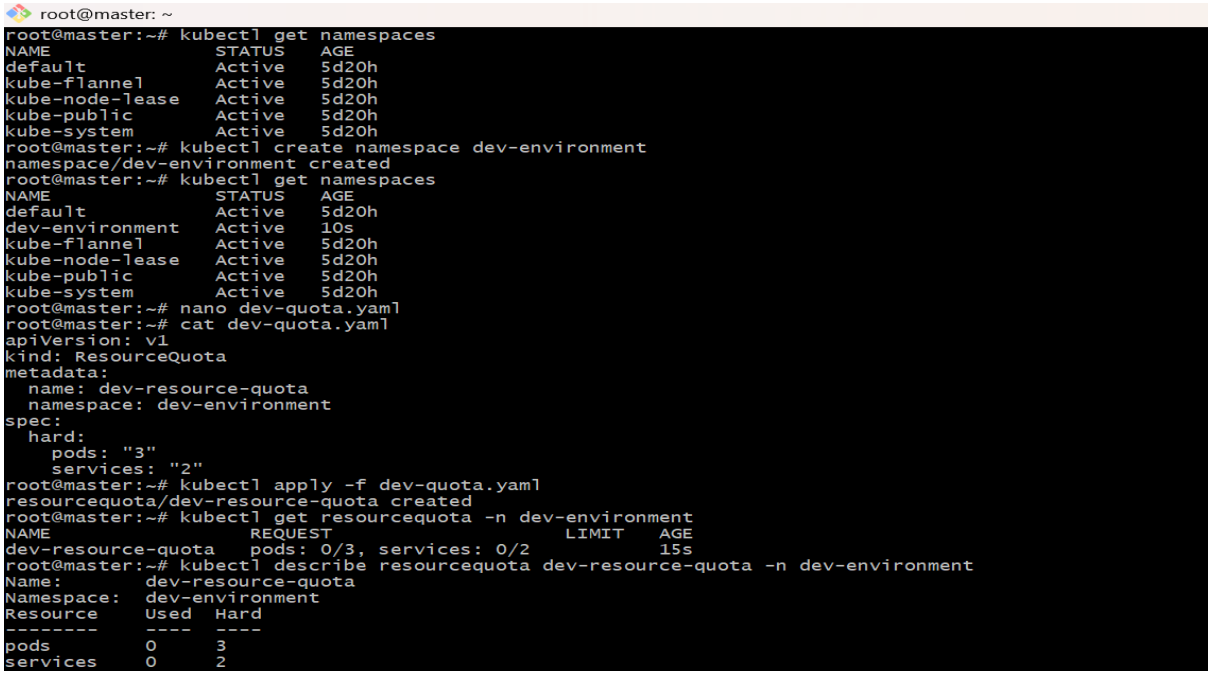
11. Try creating one more Pod:

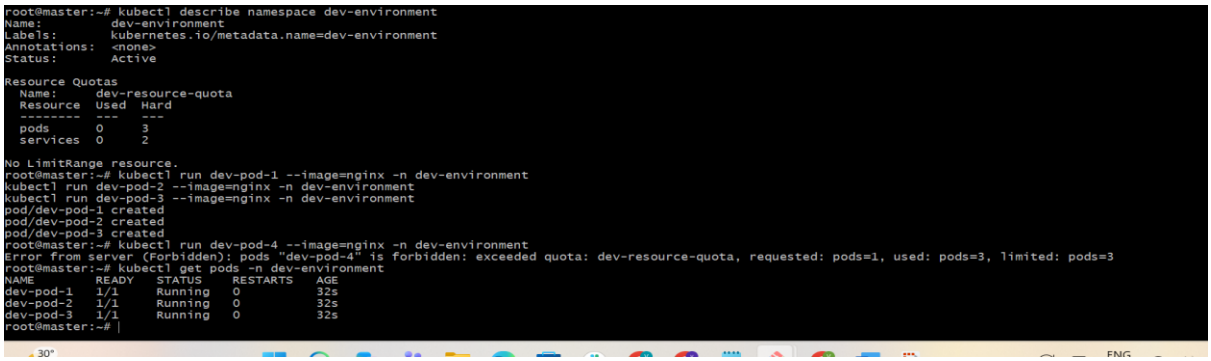
    kubectl run dev-pod-4 --image=nginx -n dev-environment

12. Verify the Pods:

    kubectl get pods -n dev-environment

Expected Result: The namespace allows a maximum of three Pods and two Services. Creating an additional Pod after reaching the quota should fail with a quota-exceeded error.





2. Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

Objective: Create a Pod with the following resource configuration:

Resource	Request	Limit
CPU	0.2 (200m)	0.5 (500m)
Memory	200Mi	500Mi

Implementation Steps:

1. Create the namespace:

   kubectl create namespace prod-environment

2. Verify the namespace:

   kubectl get namespaces

3. Create a Pod YAML file:

   vi prod-pod.yaml

4. Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
 name: prod-pod
 namespace: prod-environment
spec:
 containers:
 - name: nginx-container
  image: nginx
  ports:
  - containerPort: 80
  resources:
   requests:
    cpu: "200m"
    memory: "200Mi"
```

limits:

cpu: "500m"

memory: "500Mi"

5. Save the file and create the Pod:

   kubectl apply -f prod-pod.yaml

6. Verify the Pod:

   kubectl get pods -n prod-environment

7. Check the Pod details:

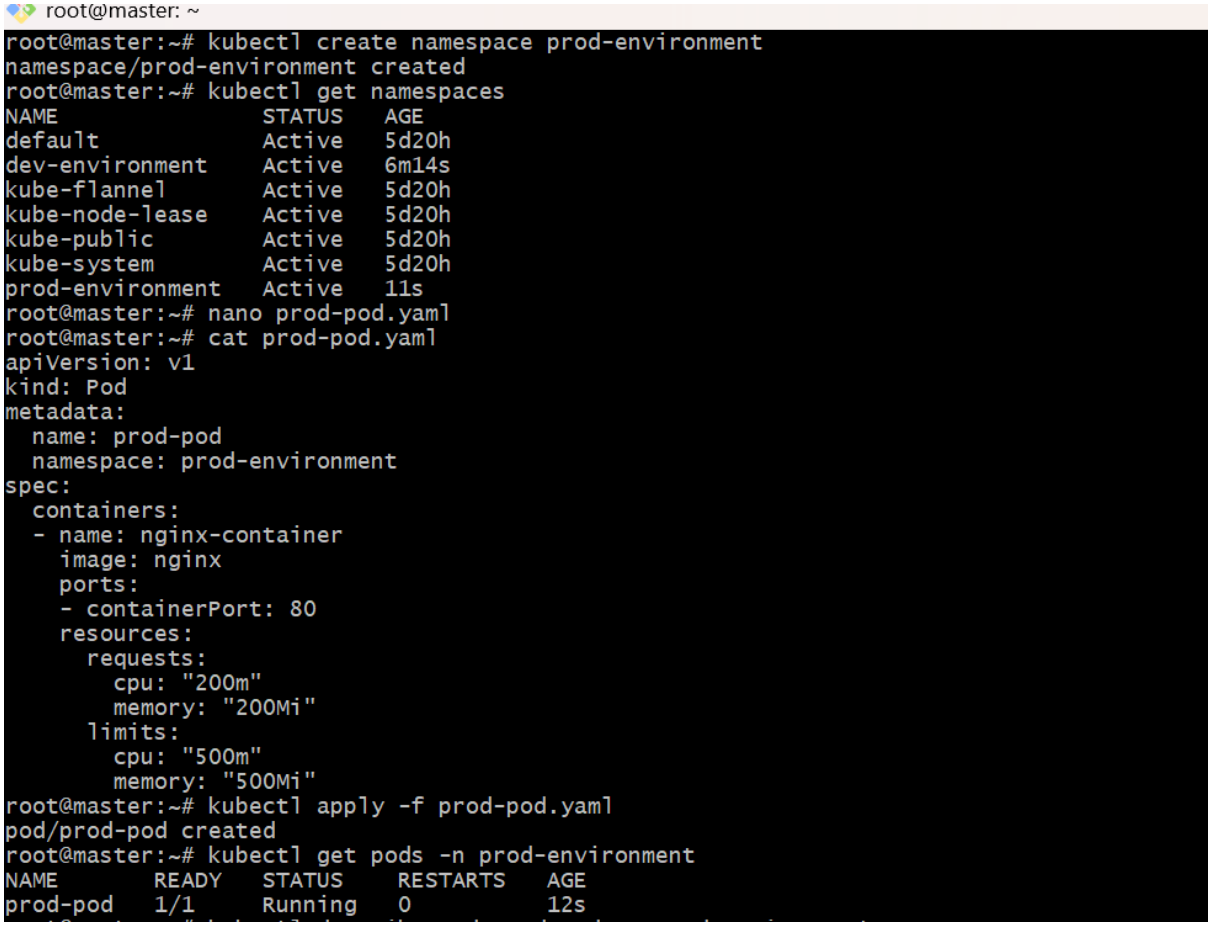
   kubectl describe pod prod-pod -n prod-environment

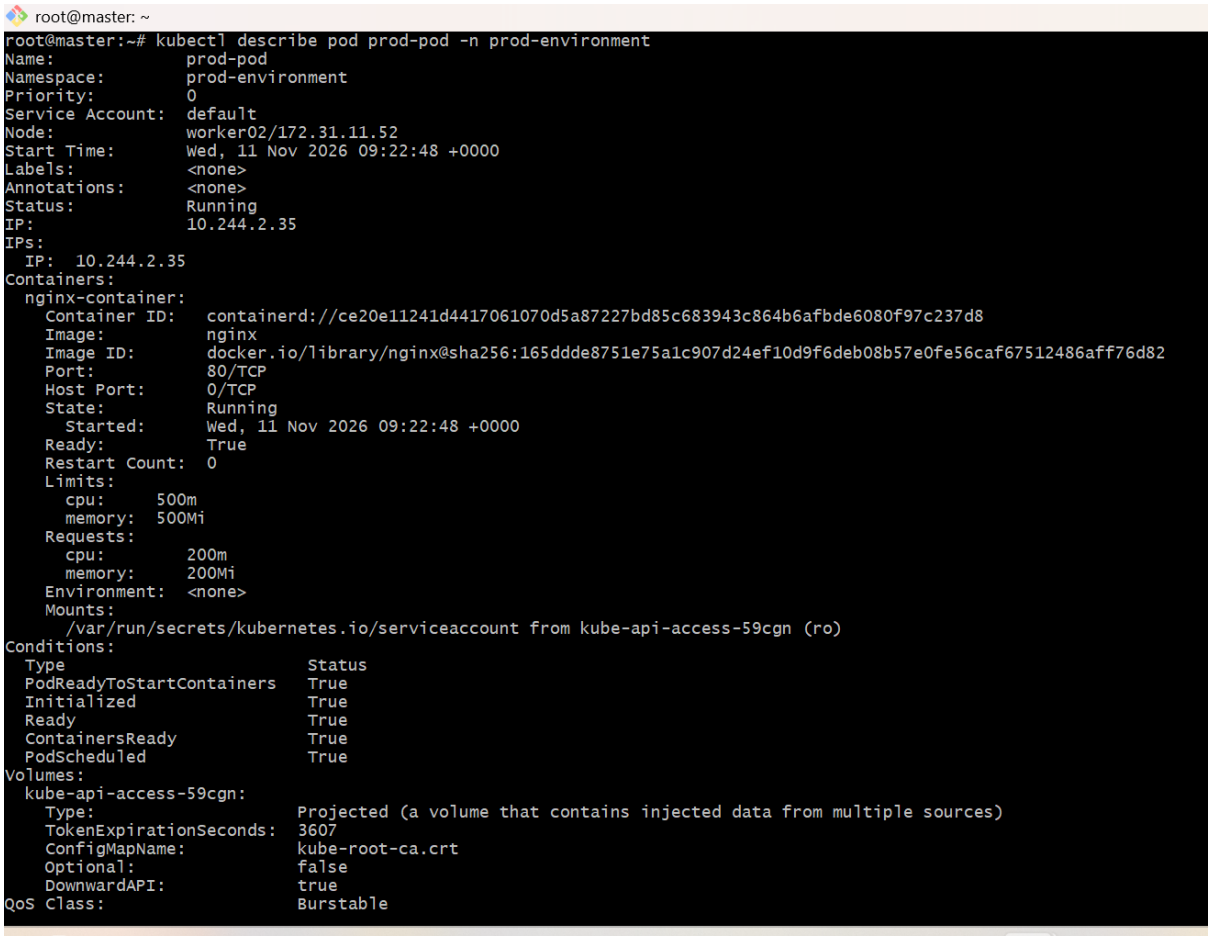
8. Verify the configured resource requests and limits in the output.

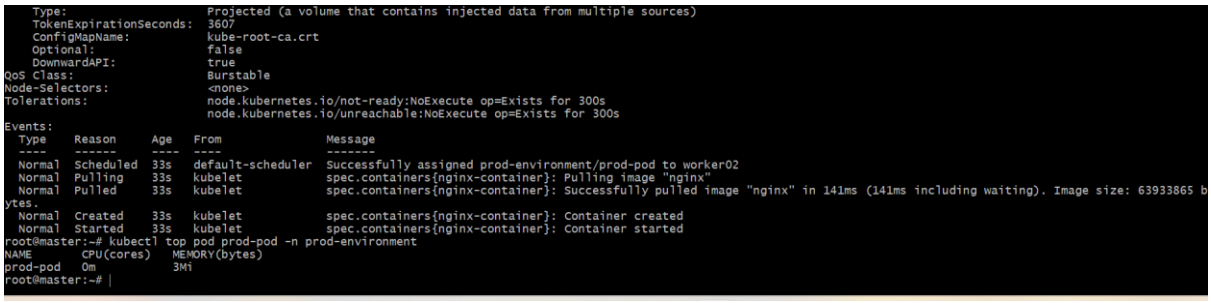
9. If Metrics Server is installed, check actual resource usage:

   kubectl top pod prod-pod -n prod-environment

   Expected Result: The Pod should be created in the prod-environment namespace with 200m CPU and 200Mi memory requests, and 500m CPU and 500Mi memory limits.







3. In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

Objective: Configure default CPU and memory limits of 300m CPU and 600Mi memory, along with minimum and maximum CPU values.

For this example, the minimum CPU is set to 100m and maximum CPU to 1 CPU because the question does not specify exact minimum and maximum values.

Implementation Steps:

1. Create the namespace:

   kubectl create namespace staging-environment

2. Verify the namespace:

   kubectl get namespaces
3. Create a LimitRange YAML file:

   vi staging-limitrange.yaml
4. Add the following configuration:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limitrange
  namespace: staging-environment
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: "600Mi"
    min:
      cpu: "100m"
    max:
          cpu: "1"
```

5. Save the file and apply the LimitRange:

   kubectl apply -f staging-limitrange.yaml
6. Verify the LimitRange:

   kubectl get limitrange -n staging-environment
7. Describe the LimitRange:

   kubectl describe limitrange staging-limitrange -n staging-environment
8. Create a Pod without specifying resource requests or limits:

   kubectl run staging-pod --image=nginx -n staging-environment
9. Verify the Pod:

   kubectl get pods -n staging-environment
10. Describe the Pod:

kubectl describe pod staging-pod -n staging-environment

11. Check the container resource configuration. Kubernetes should automatically assign the configured default CPU and memory limits.
12. Test the minimum CPU restriction by creating another Pod with a CPU request below 100m:

    kubectl run invalid-min-pod \

    --image=nginx \

    -n staging-environment \

    --overrides='{"spec":{"containers":[{"name":"nginx","image":"nginx","resources":{"requests":{"cpu":"50m"}}}]}}'

13. Test the maximum CPU restriction by creating a Pod with a CPU limit above 1 CPU:

    kubectl run invalid-max-pod \

    --image=nginx \

    -n staging-environment \

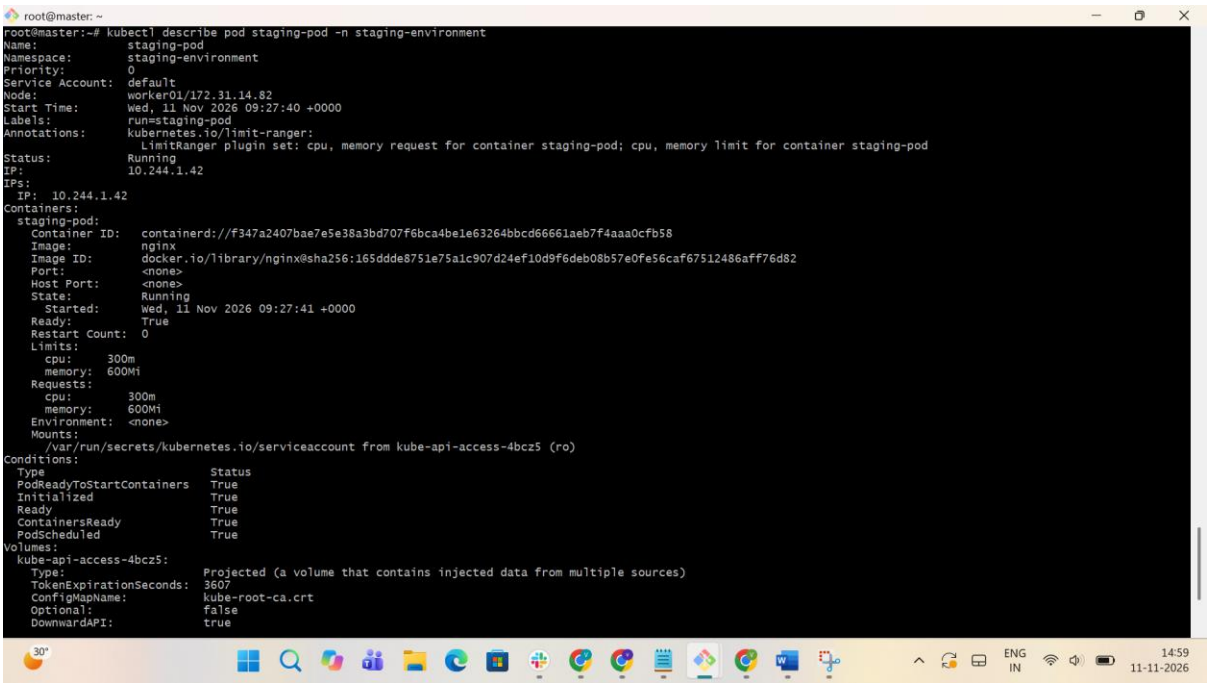
    --overrides='{"spec":{"containers":[{"name":"nginx","image":"nginx","resources":{"limits":{"cpu":"2"}}}]}}'

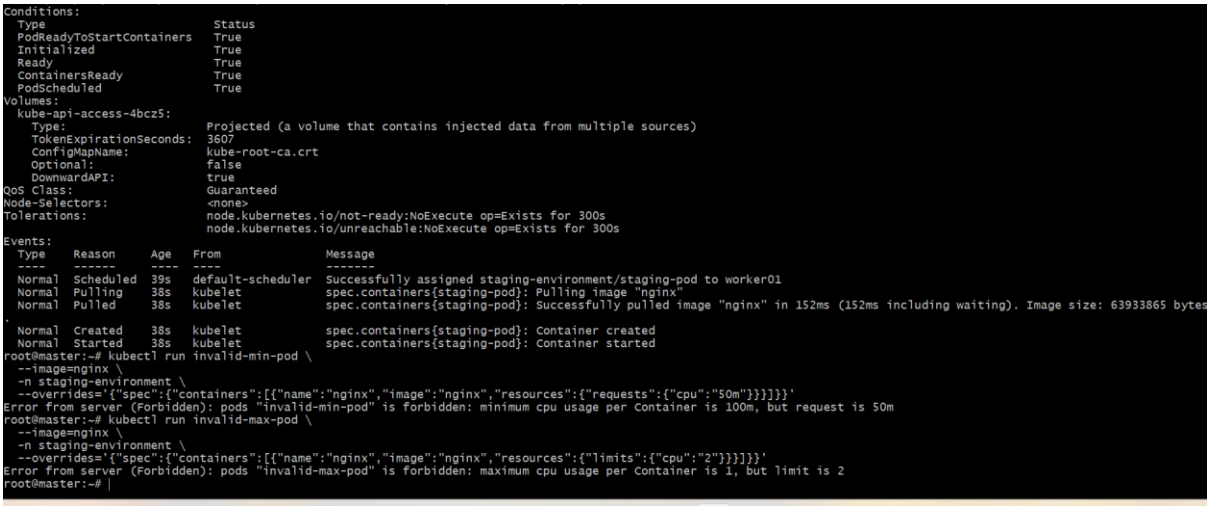
Expected Result:

- Default CPU limit: 300m.
- Default memory limit: 600Mi.
- Minimum CPU: 100m.
- Maximum CPU: 1 CPU.
- Pods violating the minimum or maximum CPU restrictions should be rejected.

root@master: ~

```
root@master:~# kubectl create namespace staging-environment
namespace/staging-environment created
root@master:~# kubectl get namespaces
NAME                  STATUS   AGE
default               Active   5d20h
dev-environment       Active   10m
kube-flannel          Active   5d20h
kube-node-lease       Active   5d20h
kube-public           Active   5d20h
kube-system           Active   5d20h
prod-environment      Active   4m32s
staging-environment   Active   13s
root@master:~# nano staging-limitrange.yaml
root@master:~# cat staging-limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limitrange
  namespace: staging-environment
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: "600Mi"
    min:
      cpu: "100m"
    max:
      cpu: "1"
root@master:~# kubectl apply -f staging-limitrange.yaml
limitrange/staging-limitrange created
root@master:~# kubectl get limitrange -n staging-environment
NAME                CREATED AT
staging-limitrange   2026-11-11T09:26:51Z
root@master:~# kubectl describe limitrange staging-limitrange -n staging-environment
Name:       staging-limitrange
Namespace:  staging-environment
Type        Resource  Min   Max  Default Request  Default Limit  Max Limit/Request Ratio
----        --------  ---   ---  ---------------  -------------  -----------------------
Container   memory    -     -    600Mi            600Mi          -
Container   cpu       100m  1    300m             300m           -
root@master:~# kubectl run staging-pod --image=nginx -n staging-environment
pod/staging-pod created
root@master:~# kubectl get pods -n staging-environment
NAME          READY   STATUS    RESTARTS   AGE
staging-pod   1/1     Running   0          24s
```





4. Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

Objective: Create a Pod and NodePort Service in the default namespace. Create another Pod in the test namespace and access the first Pod through its Service DNS name.

Implementation Steps:

1. Verify the default namespace:

   kubectl get pods

2. Create a Pod YAML file:

   vi default-pod.yaml

3. Add the following configuration:

   apiVersion: v1

   kind: Pod

   metadata:

    name: web-pod

    labels:

     app: web

   spec:

    containers:

    - name: nginx-container

      image: nginx

      ports:

```
        - containerPort: 80
```

4. Save the file and create the Pod:

   kubectl apply -f default-pod.yaml

5. Verify the Pod:

   kubectl get pods

6. Create a NodePort Service YAML file:

   vi nodeport-service.yaml

7. Add the following configuration:

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
        nodePort: 32000
```

8. Save the file and create the Service:

   kubectl apply -f nodeport-service.yaml

9. Verify the Service:

   kubectl get services

10. Describe the Service:

    kubectl describe service web-service

11. Verify that the Service is connected to the Pod:

    kubectl get endpoints web-service

12. Create the test namespace:

    kubectl create namespace test

13. Create a test Pod YAML file:

vi test-pod.yaml

14. Add the following configuration:

    apiVersion: v1

    kind: Pod

    metadata:

     name: test-client

     namespace: test

    spec:

     containers:

     - name: busybox-container

      image: busybox

           command: ["sh", "-c", "sleep 3600"]

15. Save the file and create the Pod:

    kubectl apply -f test-pod.yaml

16. Verify the Pod in the test namespace:

    kubectl get pods -n test

17. Connect to the test Pod:

    kubectl exec -it test-client -n test -- sh

18. From inside the container, execute the following command to access the Service in the default namespace:

    wget -qO- http://web-service.default.svc.cluster.local

19. Verify that the Nginx welcome page HTML is returned.

20. Exit the container:

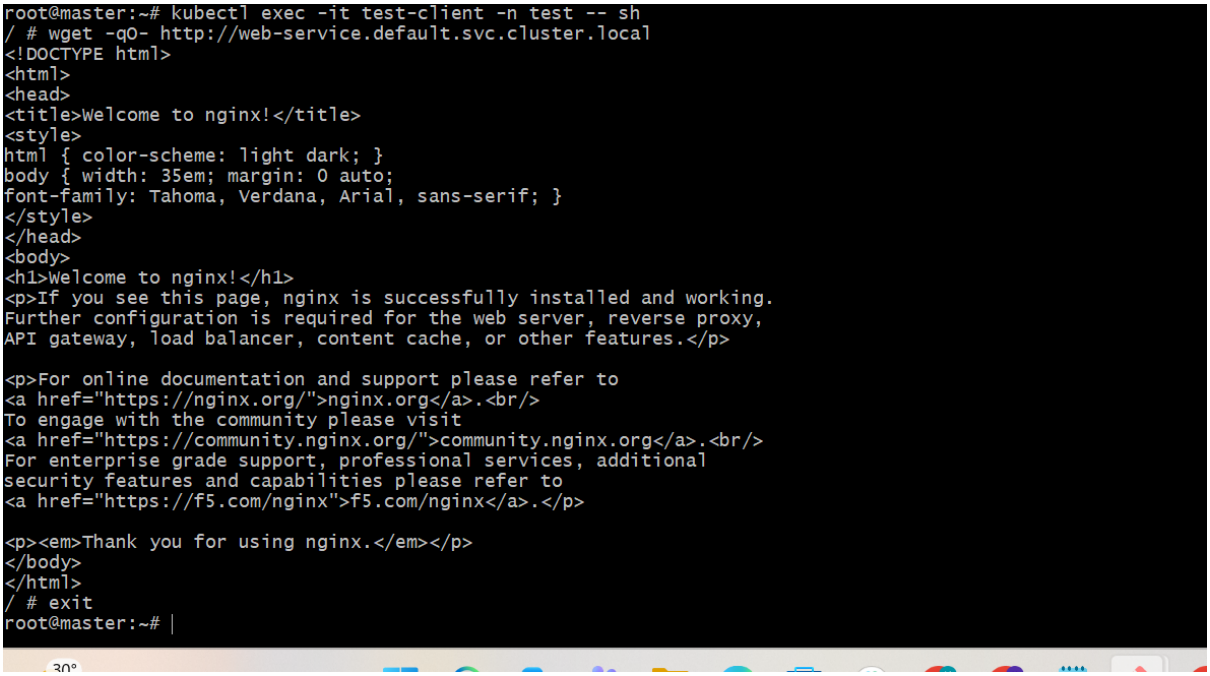
    exit

    Expected Result: The test Pod in the test namespace should successfully communicate with the Nginx Pod in the default namespace using the Service DNS name.

root@master: ~

```
root@master:~# nano default-pod.yaml
root@master:~# cat default-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
root@master:~# kubectl apply -f default-pod.yaml
pod/web-pod created
root@master:~# kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
web-pod   1/1     Running   0          15s
root@master:~# nano nodeport-service.yaml
root@master:~# cat nodeport-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 32000
root@master:~# kubectl apply -f nodeport-service.yaml
service/web-service created
root@master:~# kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          5d21h
nginx-loadbalancer  NodePort    10.97.27.98     <none>        8000:30541/TCP   44h
web-service         NodePort    10.100.53.48    <none>        80:32000/TCP     12s
```

```
root@master:~# kubectl describe service web-service
Name:                     web-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=web
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.100.53.48
IPs:                      10.100.53.48
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  32000/TCP
Endpoints:                10.244.2.36:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
root@master:~# kubectl get endpoints web-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS         AGE
web-service   10.244.2.36:80    54s
root@master:~# kubectl create namespace test
namespace/test created
root@master:~# nano test-pod.yaml
root@master:~# cat test-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-client
  namespace: test
spec:
  containers:
  - name: busybox-container
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
root@master:~# kubectl apply -f test-pod.yaml
pod/test-client created
root@master:~# kubectl get pods -n test
NAME          READY   STATUS    RESTARTS   AGE
test-client   1/1     Running   0          12s
```



5. Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

Objective: Create a LimitRange in the performance-environment namespace that restricts the memory limit-to-request ratio to a maximum of 2. Test it using a Pod with mismatched memory requests and limits.

Implementation Steps:

1. Create the namespace:

   kubectl create namespace performance-environment

2. Verify the namespace:

   kubectl get namespaces

3. Create a LimitRange YAML file:

   vi performance-limitrange.yaml

4. Add the following configuration:

   apiVersion: v1

   kind: LimitRange

   metadata:

   name: performance-limitrange

   namespace: performance-environment

```
spec:
 limits:
 - type: Container
   maxLimitRequestRatio:
         memory: "2"
```

5. Save the file and apply the LimitRange:

   kubectl apply -f performance-limitrange.yaml

6. Verify the LimitRange:

   kubectl get limitrange -n performance-environment

7. Describe the LimitRange:

   kubectl describe limitrange performance-limitrange -n performance-environment

8. Create a Pod YAML file with mismatched memory requests and limits:

   vi performance-pod.yaml

9. Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
 name: performance-pod
 namespace: performance-environment
spec:
 containers:
 - name: nginx-container
   image: nginx
   resources:
     requests:
       memory: "100Mi"
     limits:
           memory: "500Mi"
```

10. Save the file and try to create the Pod:

    kubectl apply -f performance-pod.yaml

11. Check the error message.

The ratio is:

$$
\frac{500Mi}{100Mi}=5
$$

The configured maximum ratio is 2, so Kubernetes should reject the Pod.

12. Modify the YAML file:

    vi performance-pod.yaml

13. Change the memory limit from 500Mi to 200Mi:

```
apiVersion: v1
kind: Pod
metadata:
  name: performance-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "100Mi"
      limits:
            memory: "200Mi"
```

14. Apply the corrected configuration:

    kubectl apply -f performance-pod.yaml

15. Verify the Pod:

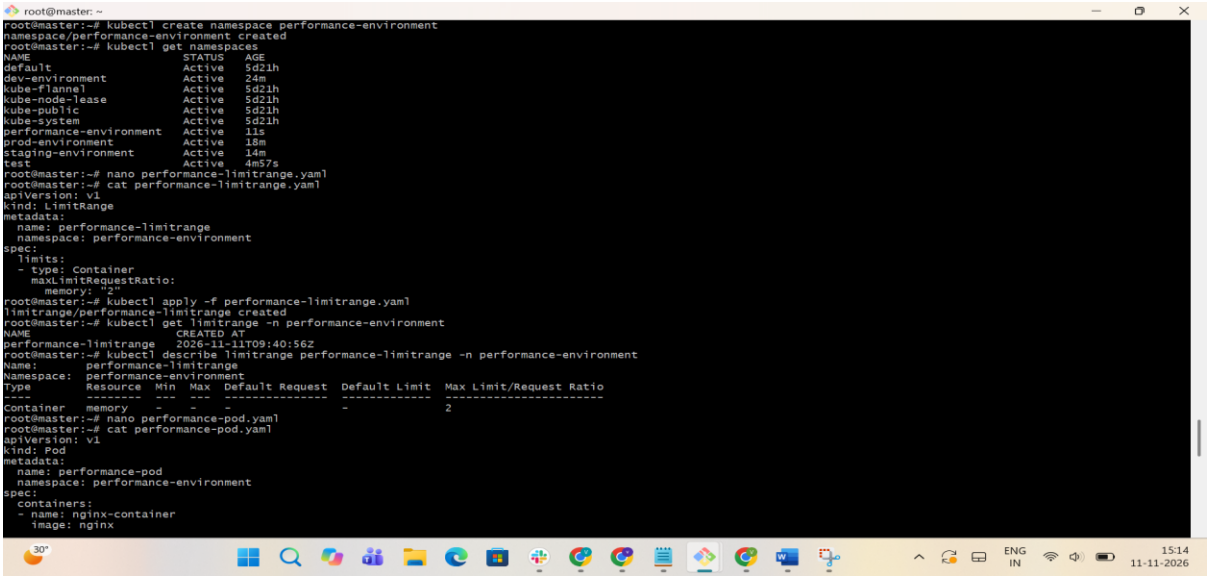
    kubectl get pods -n performance-environment

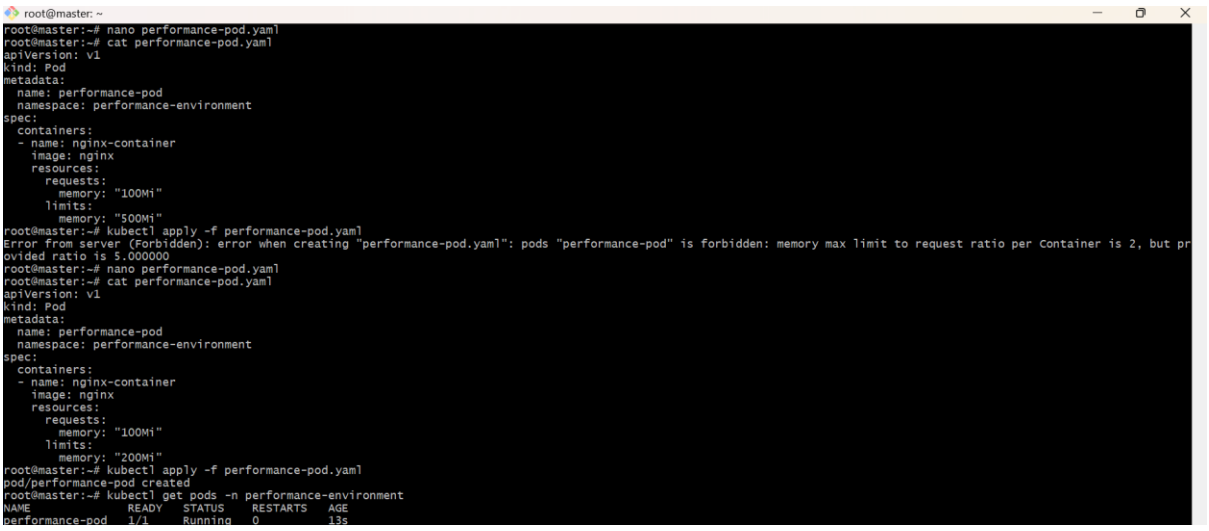
16. Describe the Pod:

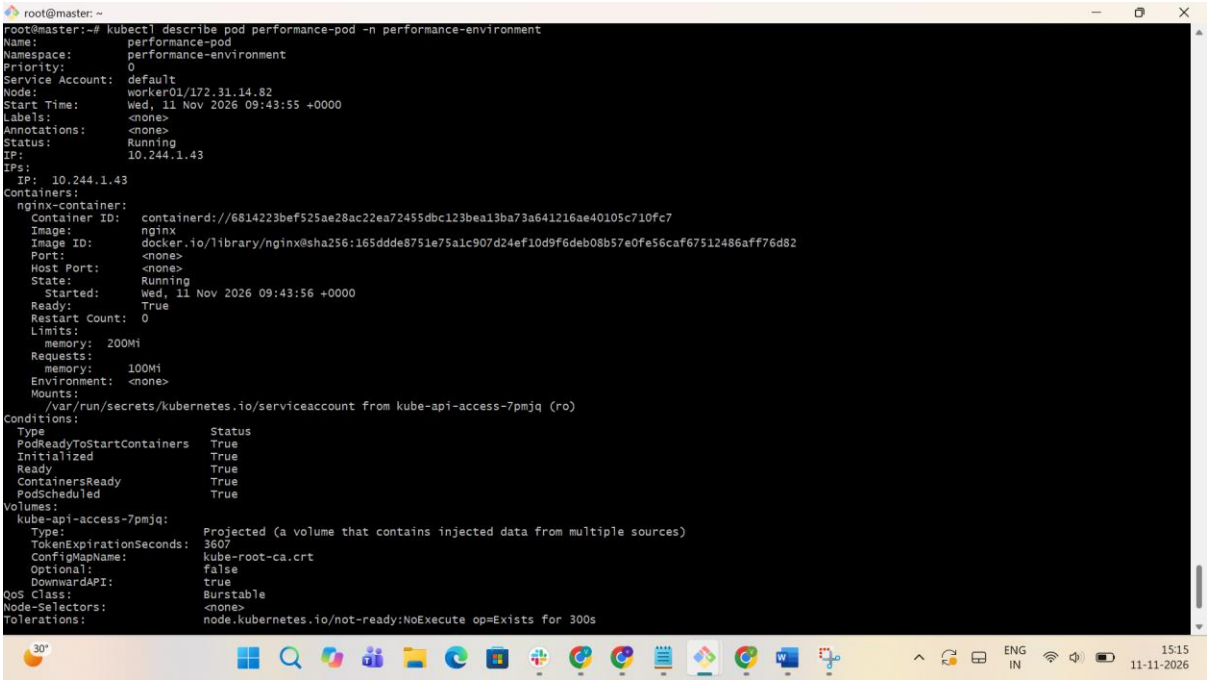
    kubectl describe pod performance-pod -n performance-environment

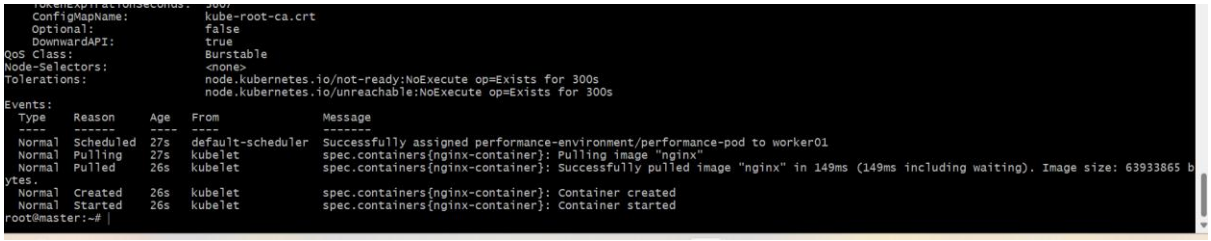
Expected Result:

- A memory request of 100Mi and limit of 500Mi should be rejected because the ratio is 5.
- A memory request of 100Mi and limit of 200Mi should be accepted because the ratio is 2.


root@master: ~
root@master:~# kubectl create namespace performance-environment
namespace/performance-environment created
root@master:~# kubectl get namespaces
NAME   STATUS   AGE
default   Active   5d21h
dev-environment   Active   24m
kube-flannel   Active   5d21h
kube-node-lease   Active   5d21h
kube-public   Active   5d21h
kube-system   Active   5d21h
performance-environment Active   11s
prod-environment   Active   18m
staging-environment   Active   14m
test   Active   4m57s
root@master:~# nano performance-limitrange.yaml
root@master:~# cat performance-limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: performance-limitrange
  namespace: performance-environment
spec:
  limits:
  - type: Container
    maxLimitRequestRatio:
      memory: "2"
root@master:~# kubectl apply -f performance-limitrange.yaml
limitrange/performance-limitrange created
root@master:~# kubectl get limitrange -n performance-environment
NAME   CREATED AT
performance-limitrange 2026-11-11T09:40:56Z
root@master:~# kubectl describe limitrange performance-limitrange -n performance-environment
Name: performance-limitrange
Namespace: performance-environment
Type   Resource Min Max Default Request Default Limit Max Limit/Request Ratio
----   -------- --- --- --------------- ------------- -----------------------
Container   memory   -   -   -   -   2
root@master:~# nano performance-pod.yaml
root@master:~# cat performance-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: performance-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx-container
    image: nginx
30°
ENG
IN
15:14
11-11-2026


root@master: ~
root@master:~# nano performance-pod.yaml
root@master:~# cat performance-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: performance-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "100Mi"
      limits:
        memory: "500Mi"
root@master:~# kubectl apply -f performance-pod.yaml
Error from server (Forbidden): error when creating "performance-pod.yaml": pods "performance-pod" is forbidden: memory max limit to request ratio per Container is 2, but pr
ovided ratio is 5.000000
root@master:~# nano performance-pod.yaml
root@master:~# cat performance-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: performance-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "100Mi"
      limits:
        memory: "200Mi"
root@master:~# kubectl apply -f performance-pod.yaml
pod/performance-pod created
root@master:~# kubectl get pods -n performance-environment
NAME   READY   STATUS   RESTARTS   AGE
performance-pod   1/1   Running   0   13s


root@master: ~
root@master:~# kubectl describe pod performance-pod -n performance-environment
Name: performance-pod
Namespace: performance-environment
Priority: 0
Service Account: default
Node: worker01/172.31.14.82
Start Time: Wed, 11 Nov 2026 09:43:55 +0000
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.1.43
IPs:
  IP: 10.244.1.43
Containers:
  nginx-container:
    Container ID: containerd://6814223bef525ae28ac22ea72455dbc123bea13ba73a641216ae40105c710fc7
    Image: nginx
    Image ID: docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Wed, 11 Nov 2026 09:43:56 +0000
    Ready: True
    Restart Count: 0
    Limits:
      memory: 200Mi
    Requests:
      memory: 100Mi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-7pmjq (ro)
Conditions:
  Type   Status
  PodReadyToStartContainers   True
  Initialized   True
  Ready   True
  ContainersReady   True
  PodScheduled   True
Volumes:
  kube-api-access-7pmjq:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
30°
ENG
IN
15:15
11-11-2026


TokenExpirationSeconds: 3607
ConfigMapName: kube-root-ca.crt
Optional: false
DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
Type Reason Age From Message
Normal Scheduled 27s default-scheduler Successfully assigned performance-environment/performance-pod to worker01
Normal Pulling 27s kubelet spec.containers{nginx-container}: Pulling image "nginx"
Normal Pulled 26s kubelet spec.containers{nginx-container}: Successfully pulled image "nginx" in 149ms (149ms including waiting). Image size: 63933865 bytes.
Normal Created 26s kubelet spec.containers{nginx-container}: Container created
Normal Started 26s kubelet spec.containers{nginx-container}: Container started
root@master:~#